

Syntrix – Personal Digital Vault

3. How to Run the Project

Purpose: Step-by-step instructions for setting up, starting and logging into the Personal Digital Vault application.

1. Prerequisites

Install the following on Windows: **.NET 8 SDK**, **Node.js/npm** compatible with Angular 19, **SQL Server LocalDB** (normally installed with Visual Studio), and **Git**. Visual Studio or VS Code is recommended.

2. Get the Project

Clone the repository and enter the project folder:

```
git clone https://github.com/viratmission/Personal-Digital-Vault.git<br>cd Personal-Digital-Vault
```

3. Backend Configuration

The backend connection string points to SQL Server LocalDB and database **PersonalDigitalVaultDB**. The project also requires an AES encryption key under **Encryption:AesKey**. The AES service requires a Base64-encoded 32-byte key (AES-256). Configure this through .NET User Secrets or another secure configuration source before running features that instantiate the encryption service.

Example User Secrets command:

```
dotnet user-secrets init<br>dotnet user-secrets set "Encryption:AesKey"
"&lt;BASE64_32_BYTE_KEY&gt;"
```

4. Restore and Prepare Database

From the repository root:

```
dotnet restore<br>dotnet ef database update
```

If the EF command is unavailable, install the EF CLI tool first with **dotnet tool install --global dotnet-ef**.

5. Run the Backend

Start the ASP.NET Core API:

```
dotnet run
```

The backend uses HTTPS redirection. In Development mode, Swagger/OpenAPI is enabled, so the Swagger UI can be opened from the HTTPS URL shown in the terminal.

6. Run the Angular Frontend

Open a second terminal and move to the frontend folder:

```
cd frontend<br>npm install<br>npm start
```

Angular's package configuration maps **npm start** to **ng serve**. Open the local Angular URL shown in the terminal (normally <http://localhost:4200>). The frontend environment uses **/api** as its API base path.

7. Login Credentials

No fixed demo/admin email and password are hard-coded in the authentication controller/service found in the repository. The supported flow is to register a new account first, then log in using that account.

Register: use the application's Register/Sign Up screen and provide Full Name, Email and Password. The backend hashes the password before storing the user. Then use the same email and password on the Login screen.

Important: Do not enter the JWT secret or AES key as the user's login password. They are application security configuration values, not user credentials.

8. Authentication Flow

The login endpoint is `POST /api/Auth/login`. A successful login returns a JWT. Protected API calls use the JWT as a Bearer token. The backend validates issuer, audience, token lifetime and signing key.

9. Quick Run Checklist

1) Start LocalDB/SQL Server → 2) Configure AES key → 3) dotnet restore → 4) dotnet ef database update → 5) dotnet run → 6) cd frontend → 7) npm install → 8) npm start → 9) Register a user → 10) Login.

Troubleshooting

Database error: verify LocalDB is installed/running and the connection string is available. AES configuration error: set `Encryption:AesKey` to a valid Base64 AES-256 key. Frontend API errors: make sure the backend is running and the frontend is using the expected API path. Port differences are normal; use the URLs printed by the terminals.

Source verification

Prepared from the repository's `PersonalDigitalVault.csproj`, `Program.cs`, `appsettings.json`, `AuthController.cs`, `AuthService.cs` and `frontend/package.json/environment.ts`.